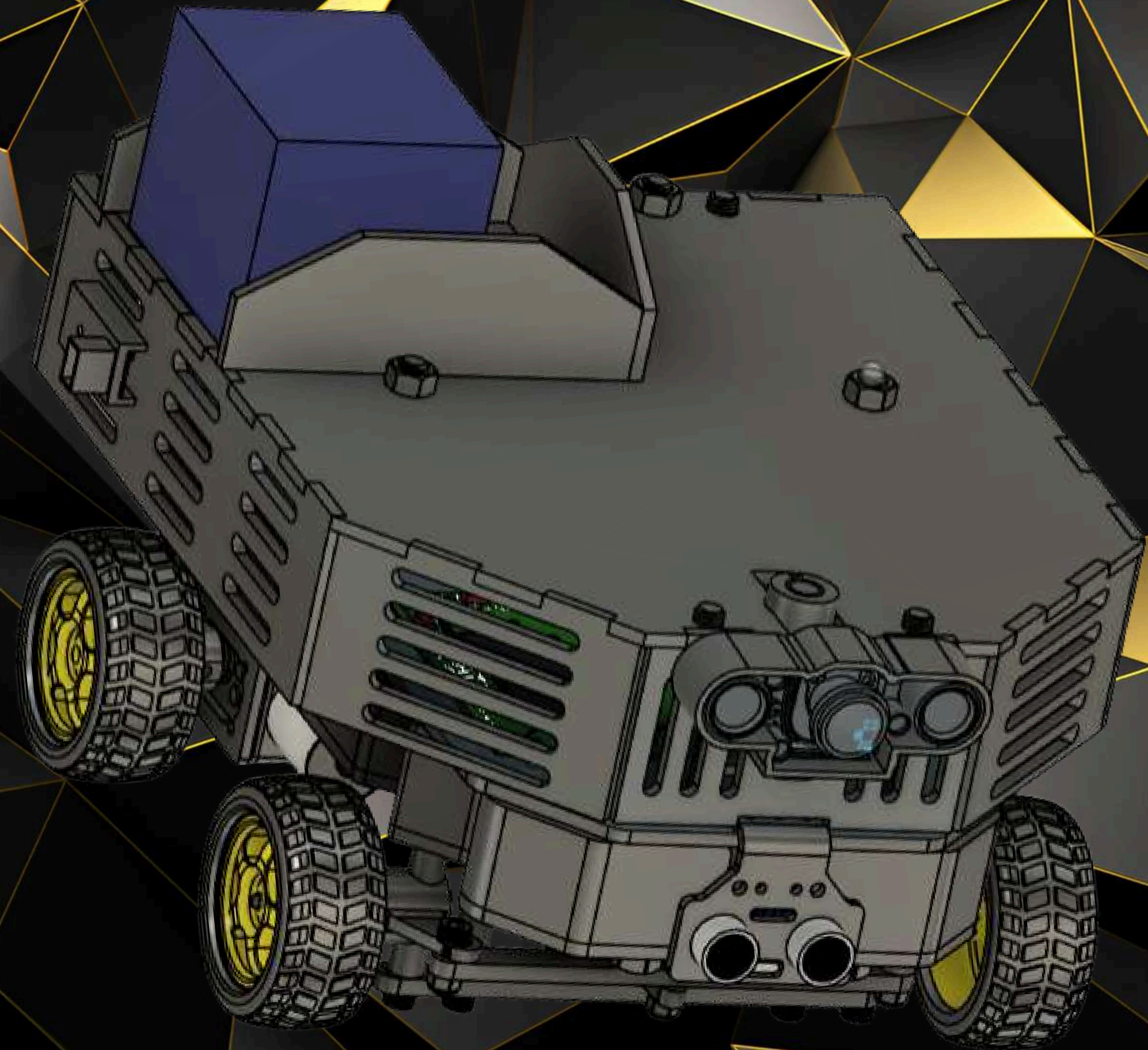




Universidad
Tecnológica de
Pereira

Presentación final ATLAS

Programa Ingeniería Mecatrónica
Facultad de Tecnologías
Pereira



Conoce al equipo detrás de ATLAS

El éxito de ATLAS es el resultado del trabajo y la dedicación de nuestro talentoso equipo de estudiantes y la invaluable guía de nuestros docentes.



Jose Lisander Hurtado
castañeda

Area de mecánica



Sofia Salomé Espitia
Jimenez

Area de Diseño 3D



Vanesa Galeano Franco

Area de programacion



Julian Andres Garcia
Correa

Area electronica



Osiel Arbelaez

Director del programa



Carlos Andres Rodriguez

Docente: proyecto integrador I



Edward Andres Gonzalez
Rios

Docente: proyecto integrador II



Tatiana Rengifo Oviedo

Docente: Formacion integral

Limitaciones actuales en el almacenamiento logístico



Los sistemas de almacenamiento convencionales dependen de operaciones manuales lentas e imprecisas, generando cuellos de botella en la cadena de suministro.

Fuente imagen

Carga y Descarga Ineficiente

Tiempos elevados y alta tasa de error humano en la manipulación de mercancías.

Falta de optimización

Rutas no planificadas y gestión reactiva que incrementan costos operativos.

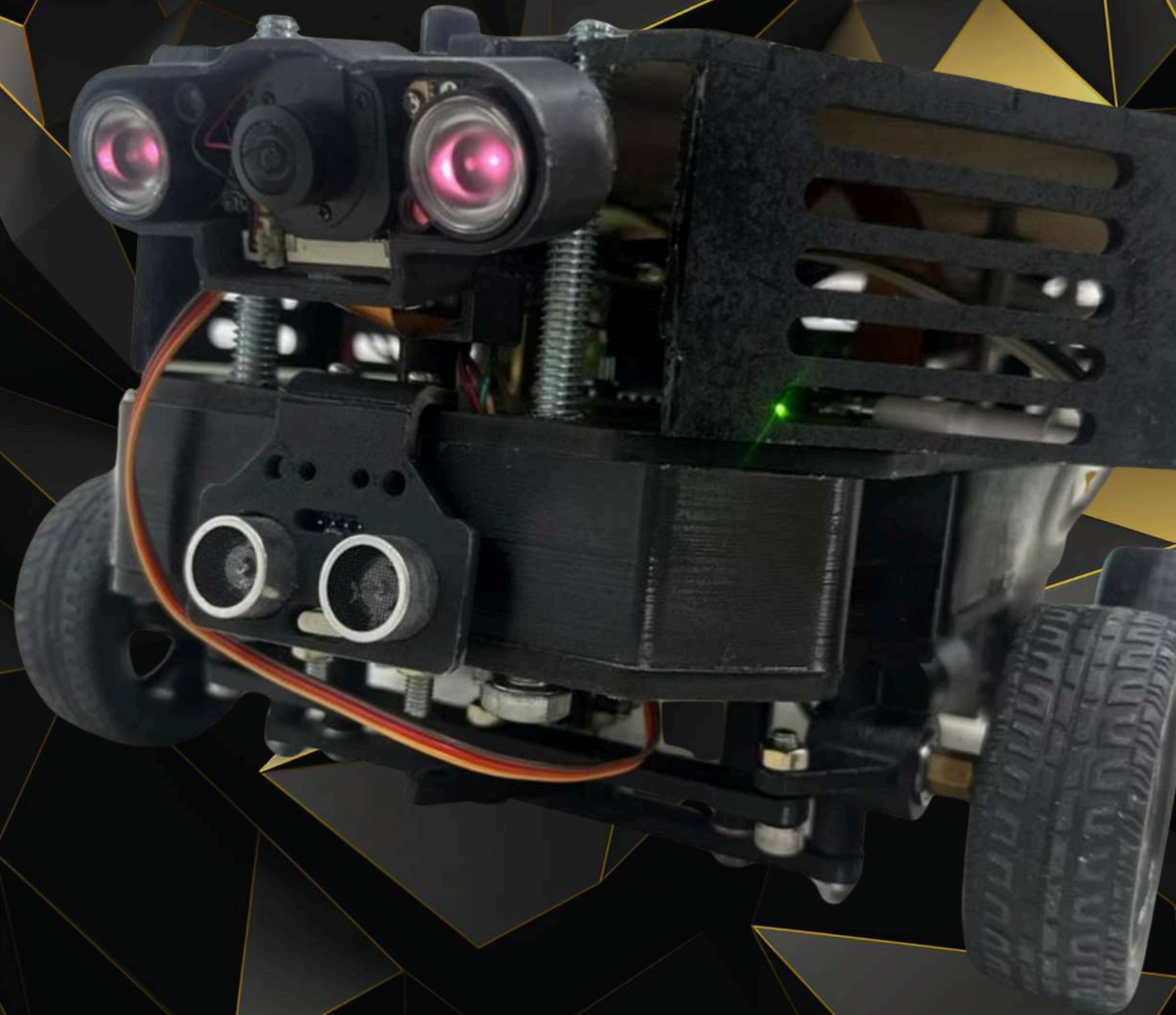
Falta de Escalabilidad

Sistemas rígidos incapaces de adaptarse al crecimiento de la demanda.

ATLAS

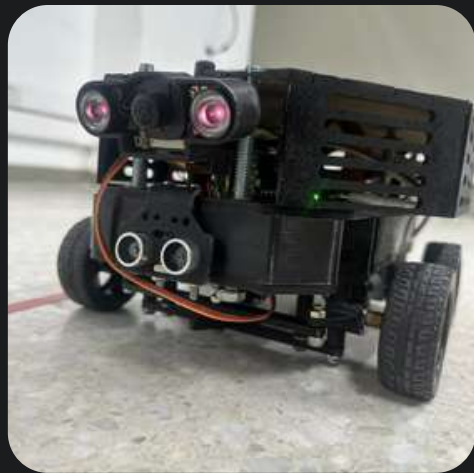
Una solución de automatización basada en robótica autónoma que optimiza la carga, descarga y distribución de mercancías mediante navegación inteligente y control de precisión.

ROBÓTICA · LOGÍSTICA · AUTOMATIZACIÓN



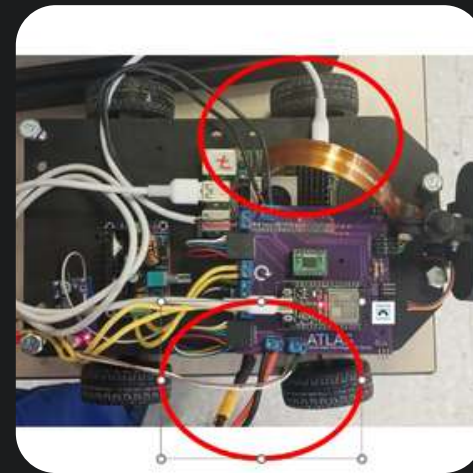
Objetivo general y objetivos específicos

Nuestro enfoque integral abarca desde la conceptualización mecánica hasta la validación en escenarios reales, garantizando una solución robótica robusta y eficiente.



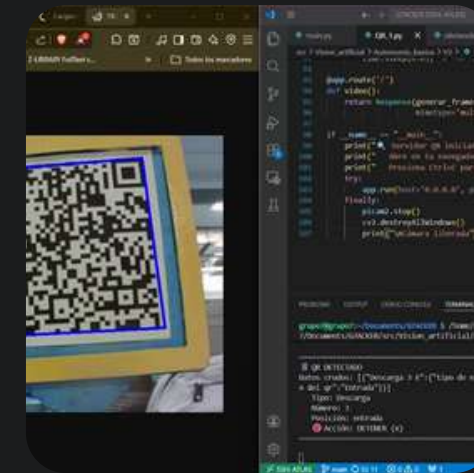
Diseño Mecánico

Concebimos la estructura física de ATLAS para una movilidad ágil y manipulación precisa de cargas en entornos logísticos.



Control y Comunicación

Desarrollamos una arquitectura híbrida de control para una coordinación fluida entre hardware y software, asegurando respuestas en tiempo real.



Visión Artificial

Implementamos sistemas de percepción avanzados para la navegación autónoma, detección de obstáculos y reconocimiento de mercancías.



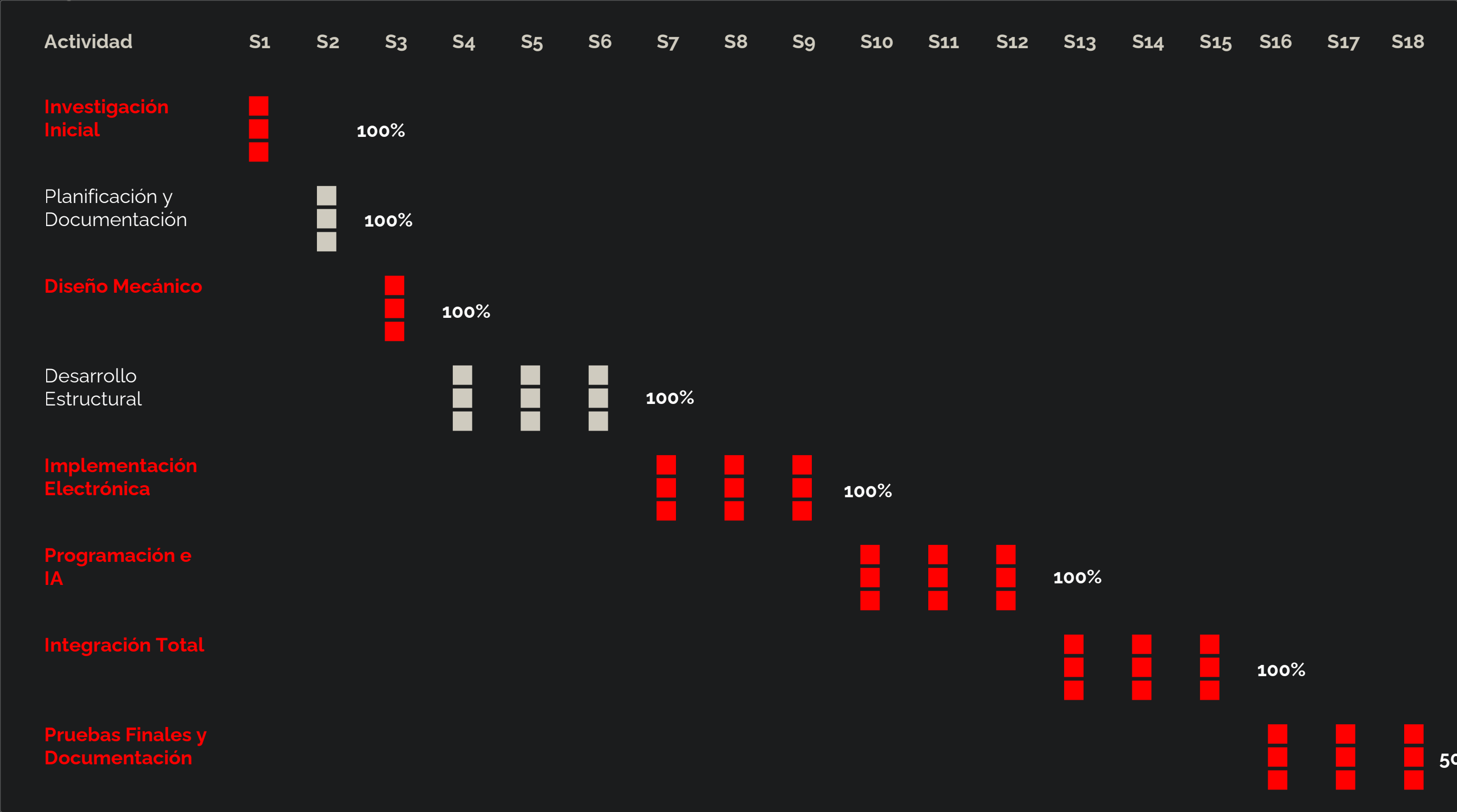
Validación del Sistema

Realizamos pruebas exhaustivas y simulaciones para garantizar el rendimiento óptimo y la fiabilidad de ATLAS en todas sus funciones.

CRONOGRAMA Y RUTA CRÍTICA – ROBOT AUTÓNOMO ATLAS

Sistema Autónomo Inteligente con Visión Artificial y Control Electrónico
Una planificación detallada es la base del éxito de ATLAS.

El siguiente cronograma visualiza las 18 semanas de desarrollo del Robot Autónomo ATLAS, destacando las actividades clave y la ruta crítica para asegurar un avance eficiente.



Las actividades destacadas en rojo brillante forman parte de la ruta crítica del proyecto, indicando tareas que no pueden sufrir retrasos sin impactar la fecha final de entrega.

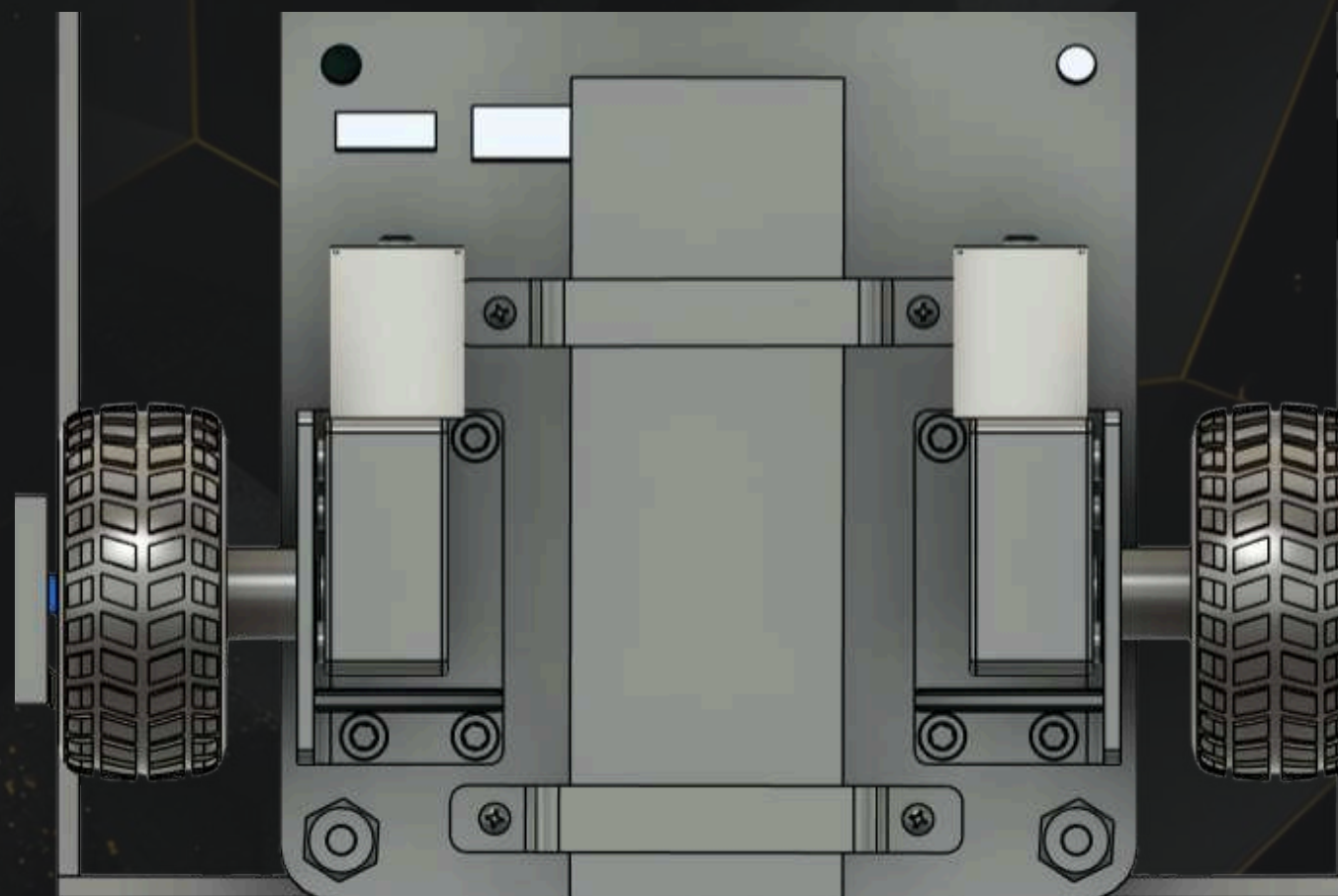
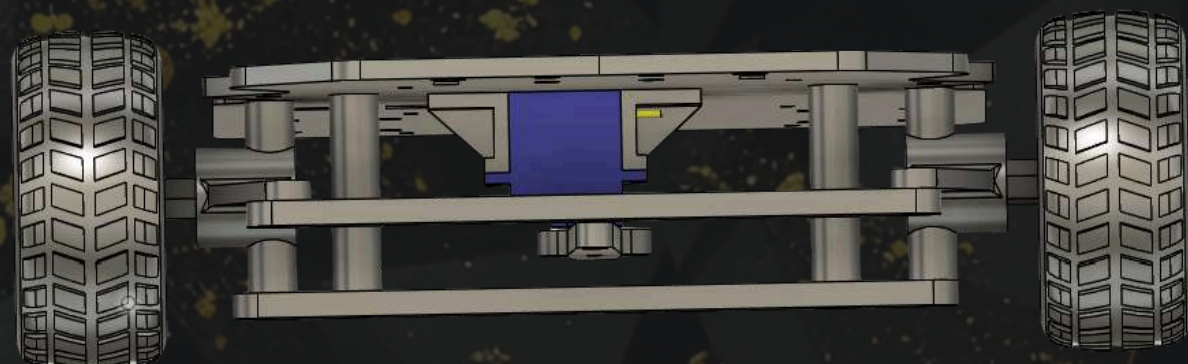
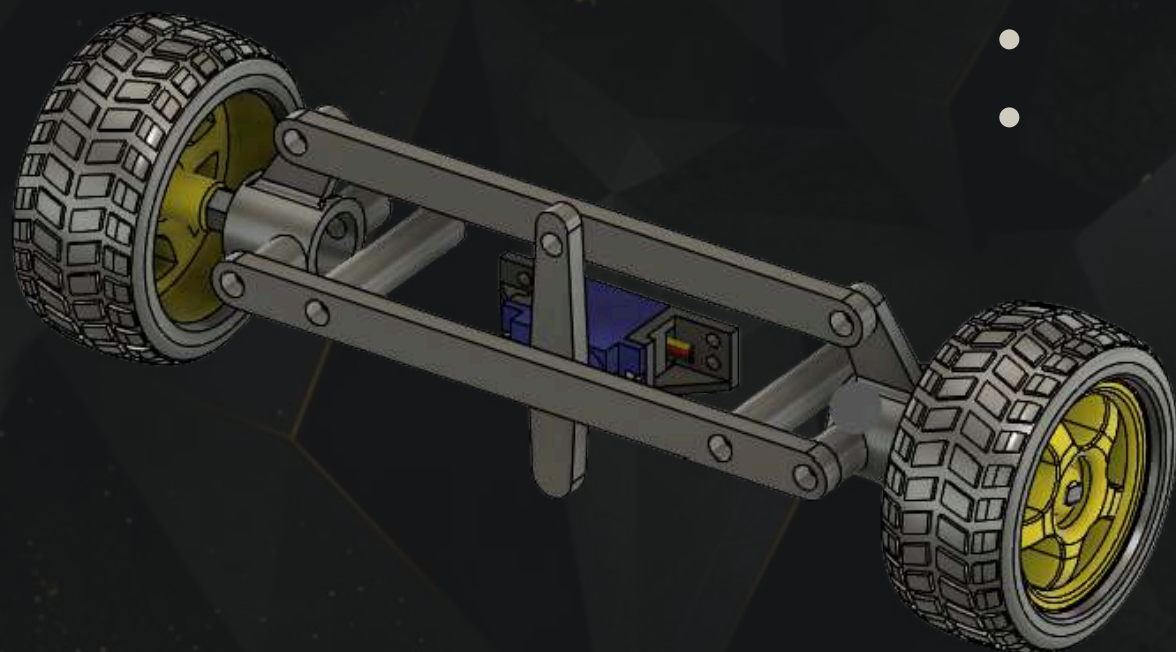
Ingeniería y Estructura de Desglose del Trabajo (EDT) de ATLAS

La complejidad de ATLAS exige un enfoque de ingeniería multifacético, donde cada disciplina contribuye a la cohesión del sistema. Nuestra Estructura de Desglose del Trabajo (EDT) organiza estas áreas clave para un desarrollo eficiente.

Mecánica

Cuenta con tres pisos:

- Nivel de Potencia
- Nivel de Electrónica
- Nivel de Carga

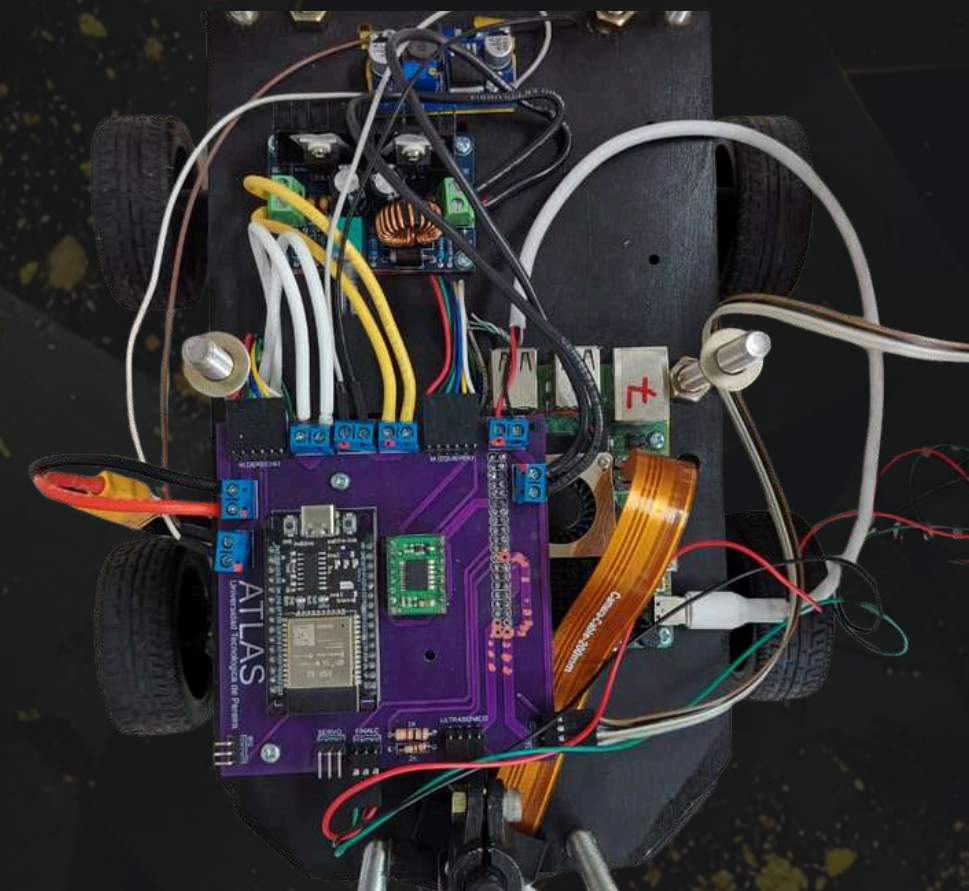
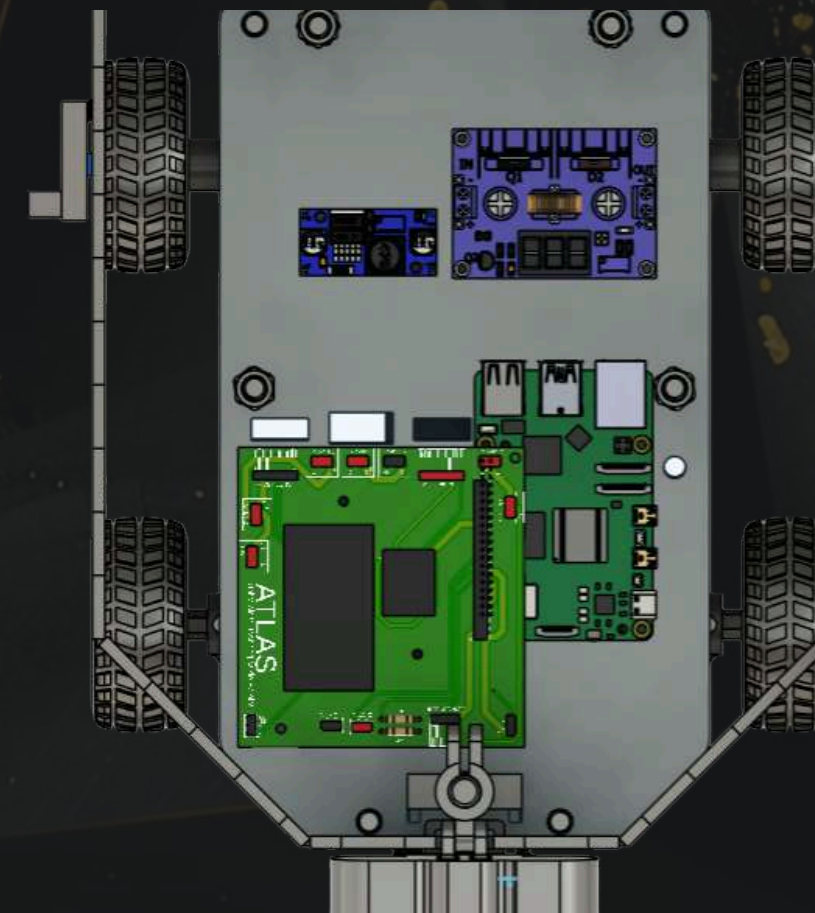
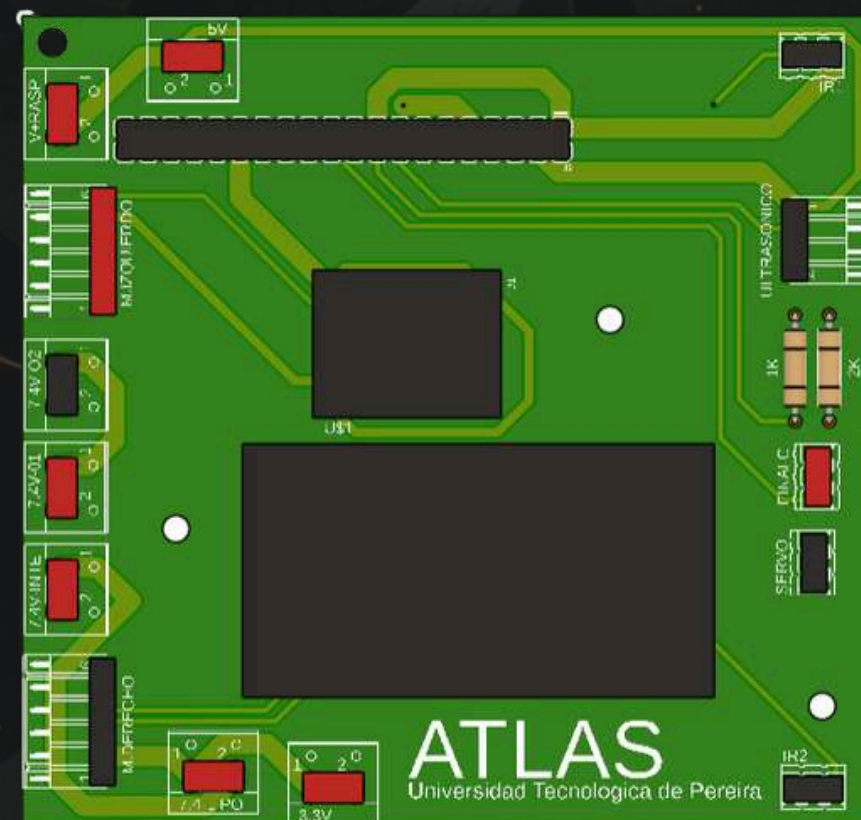
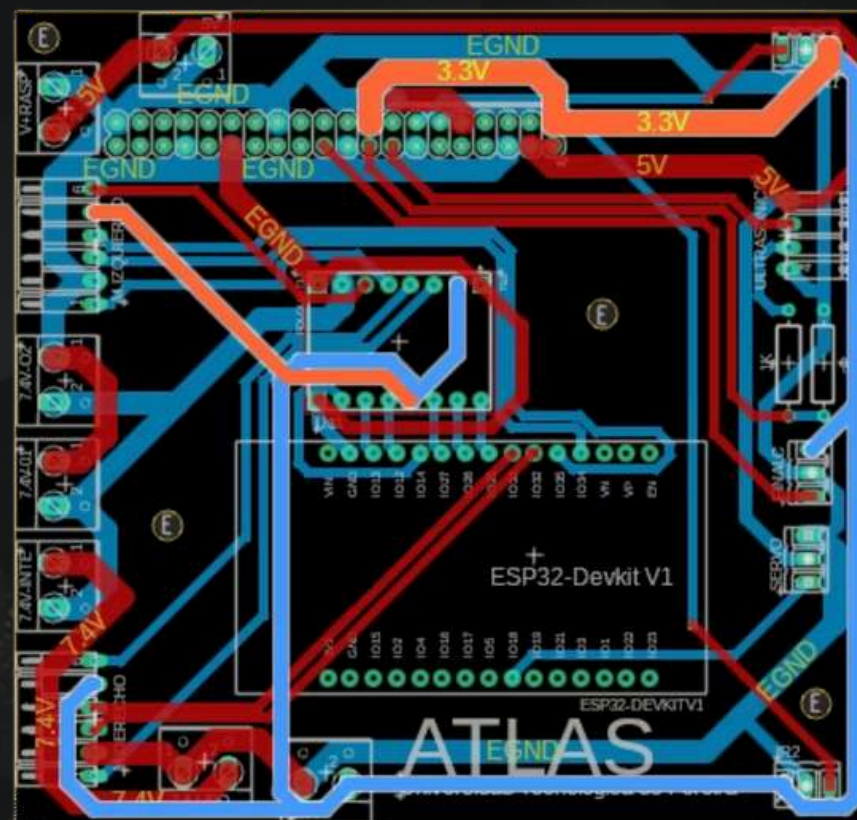


Ingeniería y Estructura de Desglose del Trabajo (EDT) de ATLAS

La complejidad de ATLAS exige un enfoque de ingeniería multifacético, donde cada disciplina contribuye a la cohesión del sistema. Nuestra Estructura de Desglose del Trabajo (EDT) organiza estas áreas clave para un desarrollo eficiente.

Electrónica

Sistemas de potencia, sensores, actuadores y subsistemas de control.



Programación

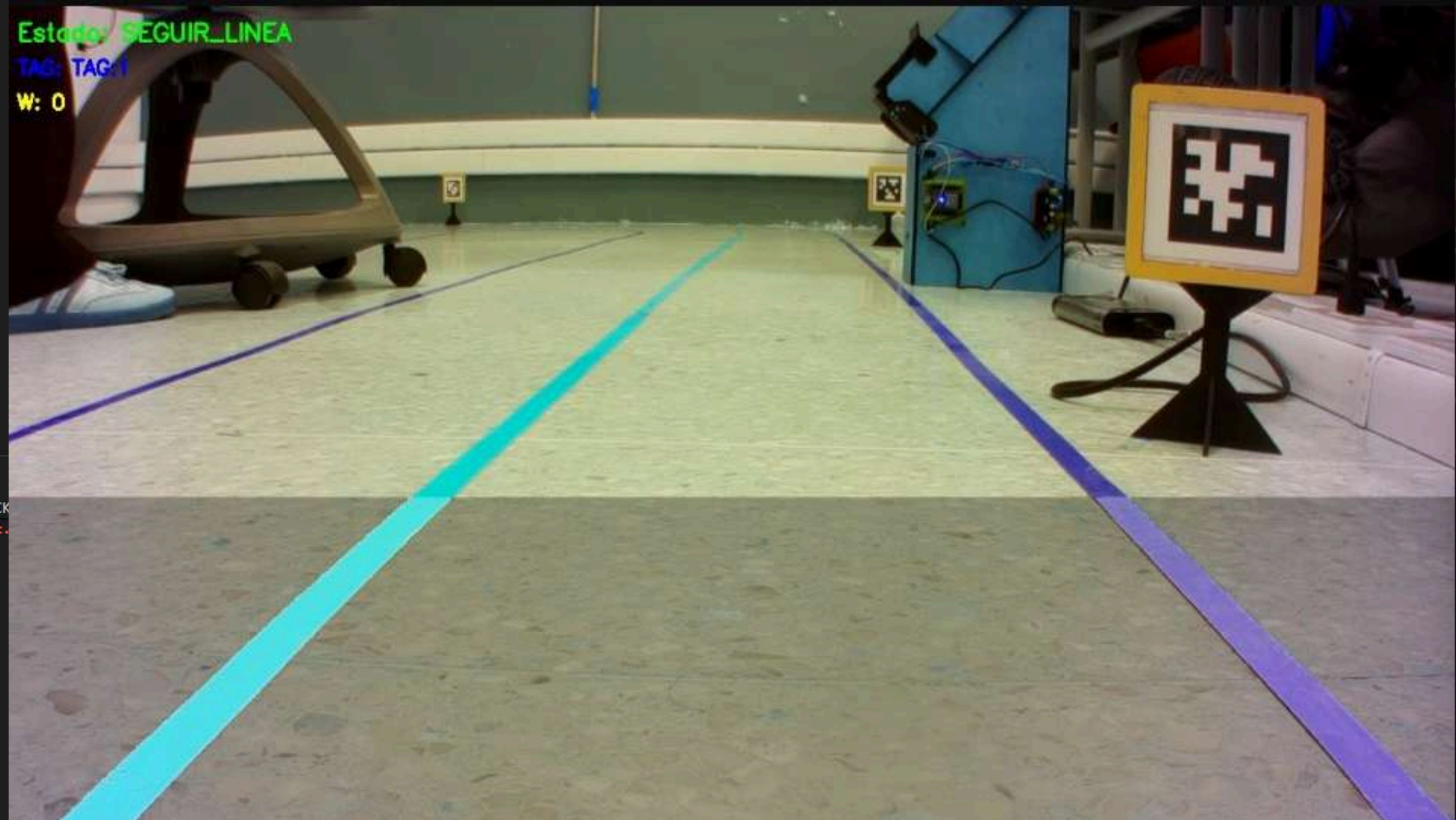
Tres codigos para el control integral del robot:

- Main.py: enlace
- Motor_control: potencia
- Qr_logic: logica

```
> hardware
> mechanical
> src
  > Autonomía_básica
    > V1
      > obstaculo_2.1.py
    > V2
      > __pycache__
      > main.py
      > motor_control.py
      > qr_logic.py
    > README.md
  > Control_de_motores
  > Pruebas
    > README.md
  > .gitignore
  > .lgd-nfy0
  > LICENSE
  > README.md
  > Scriptsactivate
  > test.jpg
```

```
5
6 import multiprocessing as mp
7 import signal
8 import sys
9
10 def signal_handler(sig, frame):
11     print("\n[Main] Cerrando...")
12     if proc_qr.is_alive():
13         proc_qr.terminate()
14         proc_qr.join()
15     sys.exit(0)
16
17 if __name__ == '__main__':
18
19     signal.signal(signal.SIGINT, signal_handler)
20
21     mp.set_start_method('spawn', force=True)
22
23     qr_queue = mp.Queue()
24     action_queue = mp.Queue()
25     line_status_queue = mp.Queue()
26     status_queue = mp.Queue()
```

```
grupo7@grupo7:~/Documents/G7ACKER $ /usr/bin/python /home/grupo7/Documents/G7ACKER/main.py
WARNING: This is a development server. Do not use it in a production deployment.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.0.102:5000
Press CTRL+C to quit
[QR Logic] Iniciado. Final de carrera GPIO25.
[Motor] Entrando a generar_frames()
[Motor] Enviado: a
[INFO] MARCO DETECTADO (w=98) -> ACERCANDO
192.168.0.111 - - [20/May/2026 21:26:23] "GET / HTTP/1.1" 200 -
[INFO] DISTANCIA OK -> DETENER
[Motor] Enviado: x
[TAG] LEÍDO: ID=1
[QR] Tag recibido: ID=1
TAG ID: 1
CONTENIDO: {'tipo': 'carga', 'numero': 1}
```



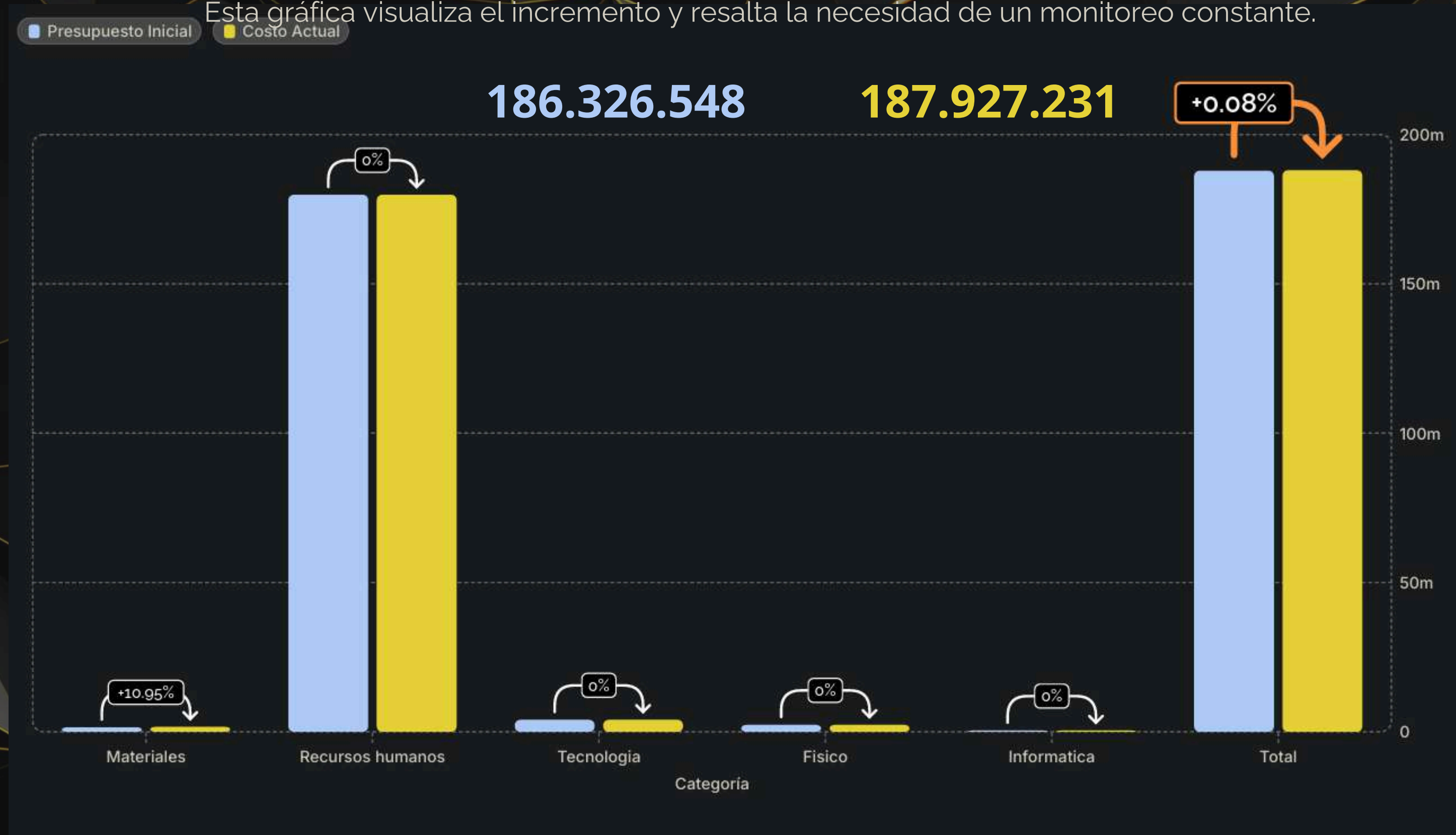
Repositorio y Documentación

Gestión de versiones, manuales técnicos y colaboración del equipo.



Gestión Financiera: Análisis de Costos de Materiales

Para evaluar la eficiencia de nuestro presupuesto, comparamos el costo inicial estimado de los materiales con los costos actuales. Esta gráfica visualiza el incremento y resalta la necesidad de un monitoreo constante.

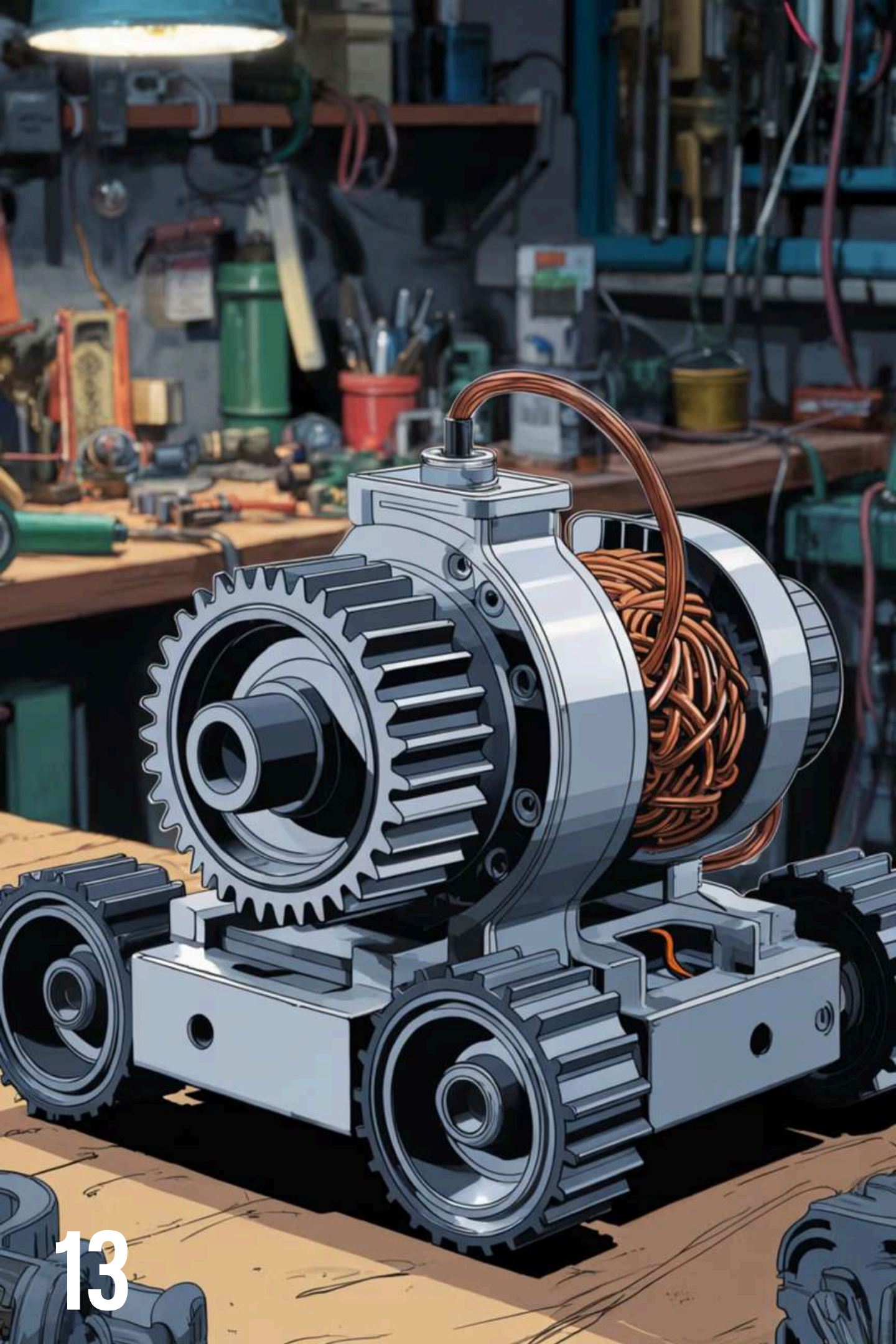


El presupuesto inicial para materiales fue de \$1,442,683, mientras que el costo actual asciende a \$1,600,683. Esto representa un incremento de +0.08% en el proyecto total, destacando la importancia de la gestión proactiva de proveedores y la optimización de compras para controlar los gastos.

Análisis de Causa Raíz: Fallas del Sistema ATLAS

Este diagrama de causa y efecto detalla los problemas potenciales que pueden afectar el rendimiento del sistema autónomo ATLAS, agrupados por categorías clave de ingeniería para facilitar la identificación y resolución.





Capacidad Operativa y Desempeño

1.5–2.5

N·m de Torque

Motores JGY-370 para transporte de hasta 1 kg de carga útil.

0.20

m/s Velocidad Nominal

Velocidad operativa controlada y estable en entornos de almacén.

0.7

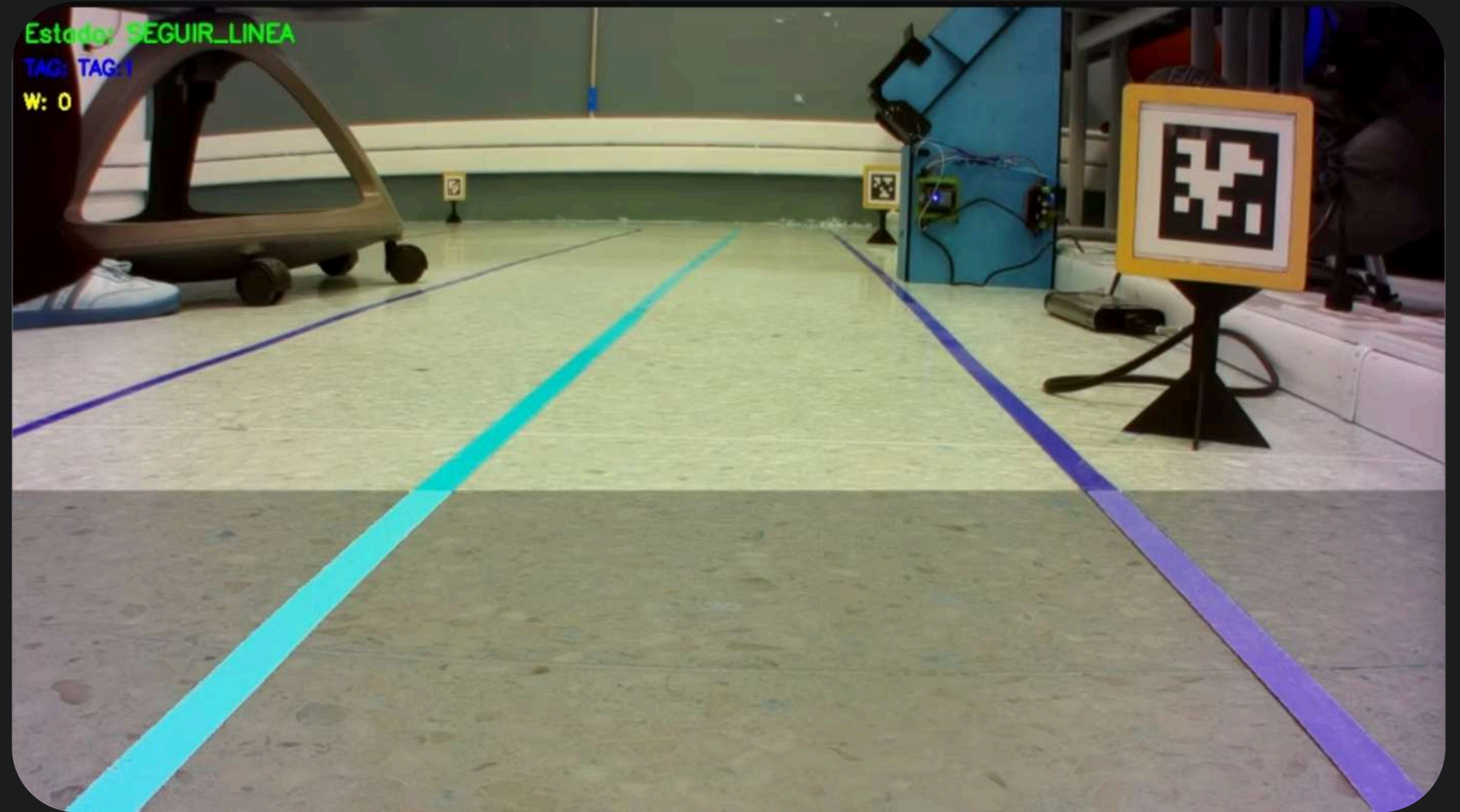
m/s Velocidad Pico

Alcanzable en tramos libres según las necesidades del entorno.

Percepción Visual

Visión Artificial con Python y OpenCV

El sistema de visión implementado procesa imágenes en tiempo real para garantizar una navegación autónoma precisa en entornos de almacén.



→ Reconocimiento de Códigos QR

Identificación de puntos de destino y zonas de carga mediante marcadores visuales estandarizados.

→ Detección de Carriles

Seguimiento de líneas guía pintadas en el suelo para navegación estructurada y sin colisiones.

→ Procesamiento en Tiempo Real

Análisis continuo del entorno visual con retroalimentación inmediata al sistema de control.

Plataforma de Investigación y Escalabilidad

El robot no es únicamente un prototipo funcional: es una **plataforma académica e industrial** diseñada para evolucionar con nuevos algoritmos y casos de uso.

Ventaja Competitiva

La combinación de **geometría Ackermann validada**, visión artificial integrada y arquitectura híbrida de control ofrece una solución replicable y escalable que supera los estándares de precisión de la industria.

Investigación Académica

Entorno controlado para probar algoritmos de percepción, SLAM y planificación de rutas en robótica educativa.

Automatización Industrial

Aplicable a sistemas de transporte interno en fábricas, almacenes y centros de distribución.

Vehículos Autónomos a Escala

Base tecnológica transferible a plataformas de mayor escala en transporte inteligente y movilidad urbana.

Conclusiones y Próximos Pasos

01

Precisión Validada

Error cinemático 17× inferior al estándar, geometría Ackermann optimizada y cumplimiento de normativa 97/27/CE.

02

Sistema Integrado y Funcional

Visión artificial, control híbrido y seguridad activa operando de forma coordinada en un único prototipo.

03

Escalabilidad y Transferencia Tecnológica

Ampliar la plataforma con SLAM, flota multi-robot y pruebas en entornos logísticos reales a mediana escala.

❏ El proyecto demuestra que la integración rigurosa de geometría mecánica, electrónica embebida y visión por computadora puede producir soluciones robóticas de alto desempeño con recursos accesibles.

Bibliografía y Referencias Clave

El proyecto ATLAS se fundamenta en una base sólida de conocimiento técnico y científico. A continuación, se presenta una selección de las referencias más relevantes que guiaron su desarrollo, organizadas por áreas de especialización.



Programación y Software

Python Software Foundation. (2024). [Python Documentation](#).

Pallets. (2023). [Flask Documentation](#).

OpenCV Organization. (2024). [OpenCV Official Documentation](#).

Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Downey, A. B. (2023). *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly Media.



Hardware y Electrónica

Raspberry Pi Foundation. (2024). [Raspberry Pi Documentation](#).

Espressif Systems. (2023). [ESP32 Technical Reference Manual](#).

Texas Instruments. (2022). *Op Amps for Everyone* (4th ed.). Elsevier.

Boylestad, R. L., & Nashelsky, L. (2021). *Electronic Devices and Circuit Theory* (13th ed.). Pearson.

HC-SR04 Ultrasonic Sensor Module Datasheet. (2023). DFRobot.



Automatización y Robótica

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2020). *Introduction to Autonomous Mobile Robots* (3rd ed.). The MIT Press.

Corke, P. I. (2017). *Robotics, Vision and Control: Fundamental Algorithms in MATLAB* (2nd ed.). Springer.

Krynicky, G. (2021). *Building Robot Applications with Python and ROS*. Packt Publishing.

Baillieul, J., & Samad, T. (Eds.). (2015). *Encyclopedia of Systems and Control*. Springer.